

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA06 20	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	Akshaya Kumar Chandrasekar	Reg. No.:	192425350
Section / Batch:	B.Tech AIML	Date:	29-07-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems.
- Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required.
- Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis.
- Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I Akshaya Kumar (192425350) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: C. Akshaya Kumar

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Bubble Sort - Reverse ordered list

Problem Interpretation

The given dataset is in reverse order

Input: [90, 75, 60, 45, 30, 15]

Why Bubble Sort

Bubble sort is a Brute force algorithm because it repeatedly compares adjacent element and swap

them

pass - by - pass sorting

Pass 1

* compare 90 & 75 - swap

* compare 90 & 60 - swap

* compare 90 & 45 = swap

* compare 90 & 30 = swap

* compare 90 & 15 - swap

Array after pass 1: [75, 60, 45, 30, 15, 90]

Pass 2

* compare 75 & 60 - swap

* compare 75 & 45 - swap

* compare 75 & 30 - swap

* compare 75 & 15 - swap

After pass 2: [60, 45, 30, 15, 75, 90]

Pass: 3

* compare 60 & 45 - swap

* compare 60 & 30 - swap

* compare 60 & 15 - swap

After pass 3: 45, 30, 15, 60, 75, 90

Pass 4:

* compare 45 & 30 - swap

* compare 45 & 15 - swap

After pass 4: [30, 15, 45, 60, 75, 90]

Pass 5

* compare 30 & 15 - swap

Array pass 5: [15, 30, 45, 60, 75, 90]

Total swaps = 15

Worst-case time complexity

* Time complexity: $O(n^2)$

* space complexity: $O(1)$

Bubble sort performs the maximum number of comparisons and swaps when input the reverse sorted.